

資料整理力

金寶電子 x Hahow for Business, 2026-06-11

郭耀仁 | yaojenkuo@ntu.edu.tw

目錄

- 什麼是整齊乾淨資料
- 任務模組一：載入資料、檢視結構
- 任務模組二：資料型態、文字處理
- 任務模組三：進階載入與資料分割
- 任務模組四：資料連接與比對
- （選修）任務模組五：寫作函數
- （選修）任務模組六：JSON 資料
- 問題與討論

今天的學習目標

1. 用 Python 讀取 Excel 活頁簿中的各張試算表（BOM）。
2. 檢視資料品質（空值、格式錯誤或重複值等）。
3. 清理並標準化資料。
4. 把 BOM 的差異用程式找出來。

什麼是整齊乾淨資料

Happy families are...

"Happy families are all alike; every unhappy family is unhappy in its own way."

-Leo Tolstoy

Tidy datasets are...

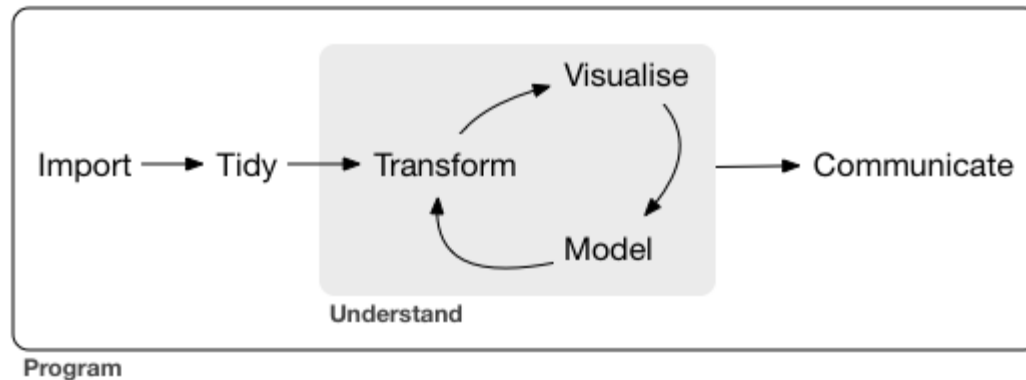
"Tidy datasets are all alike, but every messy dataset is messy in its own way."

-Hadley Wickham

整齊乾淨資料的三原則

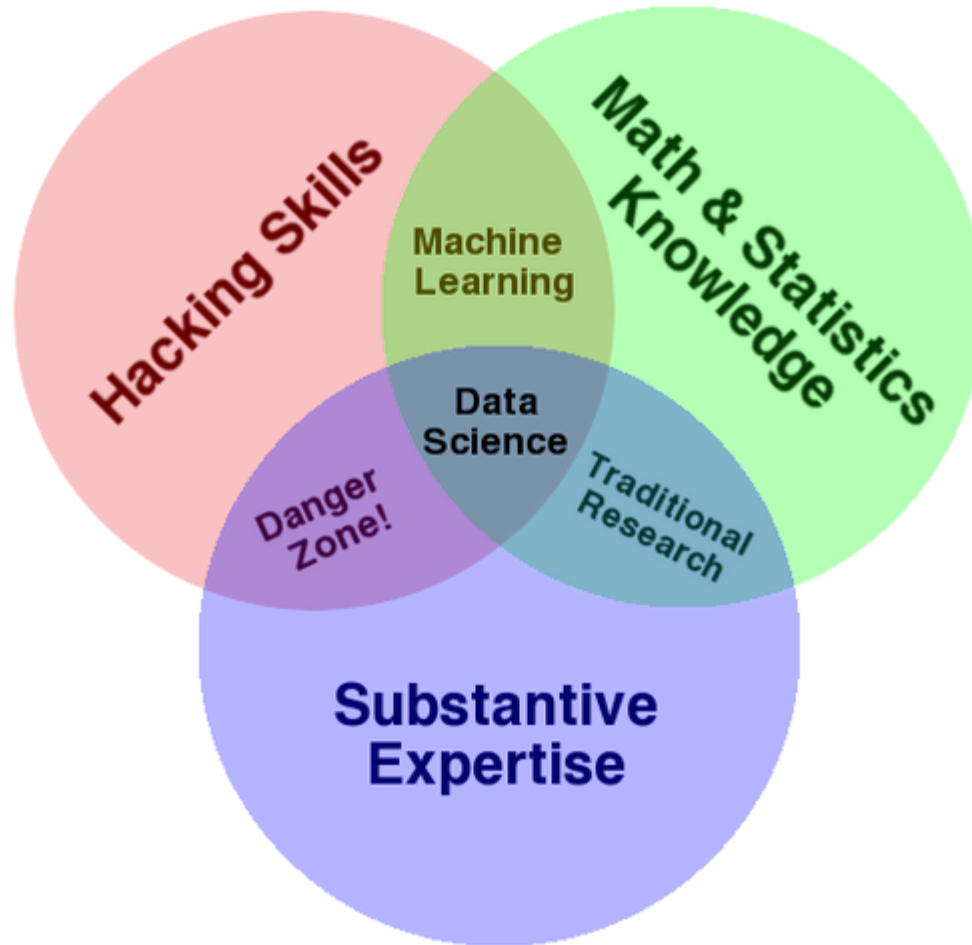
1. 每一個變數 (Variable) 形成一欄。
2. 每一觀察值 (Observation) 形成一行。
3. 每一種觀測單位形成一個表格。

現代資料科學：以程式作為資料管線（The Data Pipeline）的平台



來源：<https://r4ds.had.co.nz/introduction.html>

現代資料科學文氏圖 (Venn diagram)



來源: <http://drewconway.com/zia/2013/3/26/the-data-science-venn-diagram>

Python 資料科學模組四支柱

1. NumPy
2. Pandas
3. Matplotlib
4. Scikit-Learn

NumPy

NumPy 是 Numeric Python 的簡稱，是 Python 最重要的資料科學模組之一。NumPy 創造了 `ndarray` 的資料結構類別以及大量的通用函數與聚合函數，讓 Python 使用者能夠對進行快速的數值計算、使用統計函數、進行線性代數運算以及操作隨機的模擬任務等。

來源：<https://numpy.org/>

Pandas

Pandas 是 Python 處理表格式資料 (Tabular data) 的第三方模組，Pandas 創造了 `Index`、`Series` 與 `DataFrame` 的資料結構類別，讓 Python 在面對表格式資料時能夠用更直覺的觀念操作。

來源：<https://github.com/pandas-dev/pandas>

Matplotlib

Matplotlib 是 Python 資料視覺化的第三方模組。

來源： <https://matplotlib.org>

Scikit-Learn

Scikit-Learn 是 Python 機器學習的第三方模組，透過它可以進行監督式以及非監督式學習，提供了模型訓練、資料預處理、模型選擇以及模型評估等功能。

來源：<https://scikit-learn.org>

(沒什麼用的) 資料科學模組冷知識

- NumPy 的前身為 1990 年代中期誕生的 Numeric 與 Numarray 模組。
- Pandas 的命名由來跟熊貓「沒有關係」。
- Matplotlib 與 Matlab 「有關係」。
- Scikit 指的是 SciPy Toolkit

任務模組零：起步走

1. 前往 <https://colab.research.google.com/> 開啟新的筆記本。
2. 將 `_資料整理課程使用_MFBT_BOM.xlsx` 上傳到工作目錄（確切檔名請以課程取得為準）。
3. 載入資料整理所需的模組 `numpy` 與 `pandas`

```
In [ ]: import numpy as np
import pandas as pd

print(np.__version__)
print(pd.__version__)
```

今天要解決的 10 個問題

任務模組	#	問題
一	Q1	Specification 全是 #REF!
一	Q2	Derating Check 分類
二	Q3	Operate Current 的資料型態
二	Q4	製造商名稱大小寫不一致
二	Q5	Part Reference 拆解英文與整數
三	Q6	compare 試算表有雙層表頭
三	Q7	RefDes 資料分割
四	Q8	兩版 BOM 要比對差異
四	Q9	重複的 Part Reference
四	Q10	找出 Part Reference 末梢逗號

任務模組一：載入資料、檢視結構

任務模組一： 載入資料

_資料整理課程使用_MFBT_BOM.xlsx 活頁簿有三張試算表：

1. SCCB_32-E10384-01
2. SCCB_32-C10384-02
3. compare

使用 `pd.read_excel()` 載入資料

`sheet_name` 參數指定試算表的名稱（或位置。）

```
In [ ]: # 修改為活頁簿相對路徑
workbook_name = "_機敏資料勿外流_金寶_資料整理課程使用_for MFBT BOM_20260526"
df1 = pd.read_excel(workbook_name, sheet_name="SCCB_32-E10384-01")
df2 = pd.read_excel(workbook_name, sheet_name="SCCB_32-C10384-02")
df_compare = pd.read_excel(workbook_name, sheet_name="compare")
```

任務模組一：熱身題

在 Google Colab 載入範例活頁簿中的三個試算表，物件依序命名為 `df1`、`df2`、`df_compare`

In []: `# Your turn`

任務模組一：檢視結構

檢視資料框 (DataFrame) 的屬性：

- `df.shape`
- `df.dtypes`
- `df.columns`
- `df.index`

```
In [ ]: print(df1.shape)
        print(df1.dtypes)
        print(df1.columns)
        print(df1.index)
```

任務模組一：熱身題

分別檢視 `df2`、`df_compare` 的四個屬性。

In []: `# Your turn`

任務模組一：檢視結構

呼叫資料框 (DataFrame) 的方法：

- `df.head()`
- `df.tail()`
- `df.describe()`
- `df.info()`

```
In [ ]: print(df1.head())  
        print(df1.tail())  
        print(df1.describe())  
        print(df1.info())
```

任務模組一：熱身題

分別呼叫 `df2`、`df_compare` 的四個方法。

```
In [ ]: # Your turn
```

任務模組一：重新命名欄位名稱

```
In [ ]: import re

df1_columns = df1.columns.tolist()
df1_columns = [cn.strip() for cn in df1_columns]
df1_columns = [re.sub("\s+", "_", cn) for cn in df1_columns]
df1_columns[-2] = "Case_temperature_TA_65"
df1.columns = df1_columns
print(df1.columns)
```

任務模組一：熱身題

重新命名 `df2` 的欄位名稱。

In []: `# Your turn`

任務模組一： Q1 | Specification 欄全是 #REF!

- 檢視 Specification 欄位的缺漏值。
- 刪除 Specification 欄位。

```
In [ ]: print(df1["Specification"].isna())
print(df1["Specification"].isna().sum())
print(df1["Specification"].isna().sum() / df1.shape[0])
print(df1["Specification"].unique())
df1 = df1.drop(columns="Specification")
```

任務模組一： Q1 | Specification 欄全是 #REF!

在 df2 重現：

- 檢視 Specification 欄位的缺漏值。
- 刪除 Specification 欄位。

In []: *# Your turn*

任務模組一： Q2 | Derating Check 分類

- 檢視 `Derating_Check` 欄位的獨一值計數。
- 重新分類 `Derating_Check` 欄位。

```
In [ ]: print(df1["Derating_Check"].value_counts(dropna=False))  
        print(df1["Derating_Check"].unique())
```

```
In [ ]: import numpy as np

derating_check_dict = {
    "Pass": "Pass",
    np.nan: "Not Available",
    "Fail": "Fail"
}

df1["Derating_Check"] = df1["Derating_Check"].map(derating_check_dict)
print(df1["Derating_Check"].value_counts(dropna=False))
print(df1["Derating_Check"].unique())
```

```
In [ ]: def classify_derating_check(x):  
        if x == "Pass":  
            return x  
        elif x == "Fail":  
            return x  
        else:  
            return "Not Available"  
  
df1["Derating_Check"] = df1["Derating_Check"].map(derating_check_dict)  
print(df1["Derating_Check"].value_counts(dropna=False))  
print(df1["Derating_Check"].unique())
```

任務模組一： Q2 | Derating Check 分類

在 `df2` 重現：

- 檢視 `Derating_Check` 欄位的獨一值計數。
- 重新分類 `Derating_Check` 欄位。

In []: *# Your turn*

任務模組二：資料型態、文字處理

任務模組二： Q3 | Operate Current 的資料型態

- 檢視 `Operate_Current` 欄位的資料型態。
- 轉換 `Operate_Current` 欄位的資料型態。

```
In [ ]: print(df1["Operate_Current"].dtype)
        for data in df1["Operate_Current"]:
            if type(data) == str:
                print(data)
```

```
In [ ]: import numpy as np

def convert_dtype(x):
    if type(x) == str:
        return np.nan
    else:
        return float(x)

df1["Operate_Current"] = df1["Operate_Current"].map(convert_dtype)
print(df1["Operate_Current"].dtype)
```

任務模組二： Q3 | Operate Current 的資料型態

在 `df2` 重現：

- 檢視 `Operate_Current` 欄位的資料型態。
- 轉換 `Operate_Current` 欄位的資料型態。

In []: `# Your turn`

任務模組二： Q4 | 製造商名稱大小寫不一致

- 檢視 `manufacturer` 欄位的獨一值。
- 將 `manufacturer` 欄位替換成一致的名稱。

```
In [ ]: set_mfr1 = set(df1["manufacturer"].str.strip().str.upper().unique())
        set_mfr2 = set(df2["manufacturer"].str.strip().str.upper().unique())
        print(set_mfr1.difference(set_mfr2))
```

```
In [ ]: print(set_mfr2.difference(set_mfr1))
```

```
In [ ]: alias_dict = {
    "ON SEMI": "ON SEMICONDUCTOR",
    "TI": "TEXAS INSTRUMENTS",
    "TI (TEXAS INSTRUMENTS)": "TEXAS INSTRUMENTS",
    "VISHAY": "VISHAY/DALE INC.",
    "WE/WURTH": "WURTH ELECTRONIK",
    "TE CONN": "TE CONNECTIVITY",
    "ADI": "ANALOG DEVICES INC.",
    "ST": "STMICROELECTRONICS",
    "MICROCHIP": "MICROCHIP TECHNOLOGY",
}
df1["manufacturer"] = df1["manufacturer"].str.strip()
df1["manufacturer"] = df1["manufacturer"].str.upper()
df1["manufacturer"] = df1["manufacturer"].replace(alias_dict)
print(df1["manufacturer"].unique())
```

任務模組二：Q4 | 製造商名稱大小寫不一致

在 `df2` 重現：

- 檢視 `manufacturer` 欄位的獨一值。
- 將 `manufacturer` 欄位替換成一致的名稱。

In []: `# Your turn`

任務模組二： Q5 | Part Reference 拆解英文與整數

- 檢視 `Part_Reference` 欄位。
- 將 `Part_Reference` 欄位的英文和整數拆開來。

```
In [ ]: print(df1["Part_Reference"].unique()[:10])
        regex_pattern = "^( [A-Za-z]+ )(\d+)$"
        part_reference_parsed = df1["Part_Reference"].str.extract(
            regex_pattern)
        part_reference_parsed.columns = ["prefix", "num"]
```

```
In [ ]: def convert_str_to_int(x):  
        try:  
            return float(x)  
        except:  
            return np.nan  
part_reference_parsed["num"] = part_reference_parsed["num"].map(  
    convert_str_to_int)  
part_reference_parsed.head()
```

任務模組二： Q5 | Part Reference 拆解英文與整數

在 `df2` 重現：

- 檢視 `Part_Reference` 欄位。
- 將 `Part_Reference` 欄位的英文和整數拆開來。

任務模組三：進階載入與資料分割

任務模組三： Q6 I compare 表有雙層表頭

32-E10384-01			32-C10384-02	
Part Number	RefCnt	A_RefDes	B_RefCnt	B_RefDes
12-002490-01	1	J4	1	J4
12-003285-01	1	J1	0	
12-003306-01	1	J3	0	
12-003317-01	0		1	J8
12-003437-01	1	J2	0	
12-004352-01	1	J5	1	J5
12-004831-01	0		1	J6
12-004834-01	0		1	J2
12-004835-01	0		1	J7
12-004836-01	0		1	J3

```
In [ ]: compare = pd.read_excel(workbook_name, sheet_name="compare",  
                                skiprows=[0])  
compare.head()
```

任務模組三： Q6 | compare 表有雙層表頭

將整欄都是空值的欄位刪除。

In []: *# Your turn*

任務模組三： Q7 | RefDes 資料分割

- 檢視 A_RefDes 欄位。
- 將 A_RefDes 欄位的空格、換行符號取代為空字串，再將逗號作為分隔符分割。

```
In [ ]: print(compare["A_RefDes"].unique()[ :10])  
        print(compare["B_RefDes"].unique()[ :10])
```

```
In [ ]: A_RefDes_replaced = compare["A_RefDes"].str.replace(
        "\s|\n", "", regex=True)
A_RefDes_split = A_RefDes_rddddeplaced.str.split(",")
print(A_RefDes_split[:20])
```

任務模組三： Q7 | RefDes 資料分割

- 檢視 B_RefDes 欄位。
- 將 B_RefDes 欄位的空格、換行符號取代為空字串，再將逗號作為分隔符分割。

In []: *# Your turn*

任務模組四：資料連接與比對

任務模組四：Q8 | 兩版 BOM 要比對差異

```
df1.merge(df2, on="共同欄位", how="inner", suffixes=("_v1", "_v2"))
```

how	作用
inner	只保留兩表都有的（交集）
left	保留左表全部，沒有對應到的補 NaN
right	保留右表全部，沒有對應到的補 NaN
outer	兩表全部保留（聯集）

```
In [ ]: df1_df2_inner = df1.merge(df2, on="Part_Reference",  
                                   suffixes=("_v1", "_v2"))  
df1_df2_inner.head()
```

```
In [ ]: df1_df2_left = df1.merge(df2, on="Part_Reference",  
                                suffixes=("_v1", "_v2"), how="left")  
df1_df2_left.head()
```

任務模組四：Q8 | 兩版 BOM 要比對差異

使用 `how="outer"` 參數連接 `df1` 與 `df2`

```
df1.merge(df2, on="共同欄位", how="inner", suffixes=("_v1", "_v2"))
```

how	作用
inner	只保留兩表都有的（交集）
left	保留左表全部，沒有對應到的補 NaN
right	保留右表全部，沒有對應到的補 NaN
outer	兩表全部保留（聯集）

In []: `# Your turn`

任務模組四：Q9 | 重複的 Part Reference

- 呼叫 `duplicated()` 方法檢查重複值。

```
In [ ]: part_reference_duplicated = df1["Part_Reference"].duplicated()  
duplicate_part_reference = df1[part_reference_duplicated]  
duplicate_part_reference = duplicate_part_reference["Part_Reference"]  
df1[df1["Part_Reference"].isin(duplicate_part_reference)]
```

任務模組四：Q9 | 重複的 Part Reference

在 `df2` 重現：

- 呼叫 `duplicated()` 方法檢查重複值。

In []: `# Your turn`

任務模組四：Q10 | 找出 Part Reference 末梢逗號

- 呼叫 `str.endswith()` 方法檢查末梢逗號。

```
In [ ]: pr_endswith_comma = df2["Part_Reference"].dropna()  
pr_endswith_comma = pr_endswith_comma.str.endswith(",")  
row_endswith_comma = pr_endswith_comma[pr_endswith_comma].index  
df2.iloc[row_endswith_comma, :]
```

任務模組四：Q10 | 找出 Part Reference 末梢逗號

在 `df2` 重現：

- 呼叫 `str.endswith()` 方法檢查末梢逗號。
- 將末梢逗號清除。

```
In [ ]: # Your turn
```

(選修) 任務模組五：寫作函數

Python 提供三種機制供使用者組織程式碼

視應用範疇由小到大依序為：

- 函數 (Function) 。
- 類別 (Class) 。
- 模組 (Module) 。

什麼是函數

一段被賦予名稱的程式碼，能夠完成某一個文字處理或者數值計算任務，在使用函數之前，必須先確定這個函數在執行的環境中已經被定義妥善。

把即將要執行的程式碼組織為函數，並依序使用這些函數來完成任務

```
def function_one():  
    ...  
    return ...  
  
def function_two():  
    ...  
    return ...  
  
function_one()  
function_two()
```

(選修) 任務模組六：JSON 資料

什麼是 JSON

JSON(JavaScript Object Notation) 是一種輕量的資料交換格式，對於人類而言是容易閱讀和寫作的格式，對於電腦而言是容易解析和建立的格式，簡言之，是一種對人類與電腦都友善的純文字格式，JSON 雖然源於 JavaScript，但卻是獨立於該程式語言之外，能夠被眾多程式語言輕鬆解析和建立的一種理想資料交換格式。JSON 可能由兩種資料結構組成：鍵值對應關係與有序列表，可以用 Python 的 dict 與 list 來理解。

來源：<https://www.json.org/json-en.html>

使用標準模組 `json` 的 `load()` 函數載入

- 以 `open()` 函數搭配 `with` 敘述開啟 JSON 資料。
- 使用 `json.load()` 函數將檔案內容載入。
- 因為 JSON 可能由兩種資料結構組成，輸出物件可能是 `dict` 或 `list` 類別的實例。

```
import json

with open("PATH/TO/FILE") as file:
    json_object = json.load(file)
print(type(json_object)) # dict / list
```

將資料期間曾經產生虧損的公司剔除掉

- 使用的金融數據為 [datasets](#) 中的
`gross_profits_and_operating_margins.json`
- 金融數據擷取來源: [Yahoo! Finance](#)

對於「虧損」的接受度

- 假如我們對於「虧損」的接受度比較寬鬆，意即我不想在這個條件上剔除掉太多公司，可以用「毛利（Gross profit）大於零」定義為沒有虧損。
- 假如我們對於「虧損」的接受度比較嚴苛，意即我希望在這個條件上盡可能剔除公司，可以用「淨利（Net income）大於零」定義為沒有虧損。

$$\text{Gross profit} = \text{Revenue} - \text{Cost of goods sold}$$

$$\begin{aligned} \text{Net income} = & \text{Gross profit} - \text{Operating expenses} - \\ & \text{Depreciation and amortization} - \text{Taxes and other expenses} \end{aligned}$$

解題思路

- 首先將 `gross_profits_and_operating_margins.json` 上傳至 Google Colab 的工作目錄。
- 載入 `json` 模組。
- 使用 `with` 敘述搭配 `json.load()` 函數載入為 `list`
- 使用 `for` 迴圈走訪 200 個 `dict`
- 判斷 `gross_profit` 或 `net_income` 這兩個鍵所對應的多個值中是否有一個小於零（以 `net_income` 為例）。
- 將判斷結果加總，加總為 0 的紀錄為 `True`，反之紀錄為 `False`，儲存在一個新的 `dict` 之中。

```
In [ ]: import json
import numpy as np

file_path = "gross_profits_and_operating_margins.json"
with open(file_path, "r") as f:
    gross_profits_and_operating_margins_json = json.load(f)
```

```
In [ ]: is_profitable = list()
for data in gross_profits_and_operating_margins_json:
    ticker = data["ticker"]
    gross_profit = data["gross_profit"]
    net_income = data["net_income"]
    if net_income is None:
        dict_to_append = {
            "ticker": ticker,
            "is_profitable": False
        }
        is_profitable.append(dict_to_append)
    else:
        list_net_income_values = list(net_income.values())
        arr = np.array(list_net_income_values)
        arr_is_negative = arr < 0
        arr_sum = arr_is_negative.sum()
        dict_to_append = {
            "ticker": ticker,
            "is_profitable": not bool(arr_sum)
        }
        is_profitable.append(dict_to_append)
```

問題與討論

恭喜你完成了今天的課程！

今天解決的 10 個問題

任務模組	#	問題
一	Q1	Specification 全是 #REF!
一	Q2	Derating Check 分類
二	Q3	Operate Current 的資料型態
二	Q4	製造商名稱大小寫不一致
二	Q5	Part Reference 拆解英文與整數
三	Q6	compare 試算表有雙層表頭
三	Q7	RefDes 資料分割
四	Q8	兩版 BOM 要比對差異
四	Q9	重複的 Part Reference
四	Q10	找出 Part Reference 末梢逗號

下一步？

1. `df.groupby()`：分組聚合摘要。
2. `pd.melt()` / `df.pivot()`：資料的長寬轉換。
3. `matplotlib.pyplot/plotly`：視覺化。
4. 寫作為腳本檔案 `.py`：自動化，每次有新的 Excel 活頁簿可以直接執行腳本檔案。

